the device after logging in. The API defined by PKCS#11, known as Cryptoki, provides operations in the C programming language for logging into the HSM, listing available keys, and performing cryptographic operations.
Unlike a purely software KMS, an HSM is designed to offer protection against an attacker with physical access to the device. For example, the circuitry of the HSM may be encased in tough resin with embedded sensors that can detect anybody trying to tamper with the device; in which case the secure memory is wiped to prevent compromise. The US and Canadian governments certify the physical security of HSMs under the FIPS 140-2 certification program, which offers 4 levels of security: level 1 certified devices offer only basic protection of key material, while level 4 offers protection against a wide range of physical and environmental threats. On the other hand, FIPS 140-2 offers very little validation of the quality of implementation of the algorithms running on the device, and some HSMs have been found to have serious software security flaws. Some cloud KMS providers can be configured to use FIPS 140-2 certified HSMs for storage of keys, usually at an increased cost. However, most such services are already running in physically secured data centers, so the additional physical protection is usually unnecessary.

---

A KMS can be used to encrypt credentials that are then distributed to services using Kubernetes secrets. This provides better protection than the default Kubernetes configuration and enables the KMS to be used to protect secrets that aren’t cryptographic keys. For example, a database connection password can be encrypted with the KMS and then the encrypted password is distributed to services as a Kubernetes secret. The application can then use the KMS to decrypt the password after loading it from the disk.

## *POP QUIZ*

Answers are at the end of the chapter.

10. Which of the following are ways that a Kubernetes secret can be exposed to pods?